ESCUELA POLITÉCNICA
SUPERIOR DE CÓRDOBA
Universidad de Córdoba

Introducción al Aprendizaje Automático

Grado en Ingeniería Informática

Cuaderno de Laboratorio - Práctica 5: Arquitectura de Redes Neuronales

Autores:

Alicia Muriel Fernández
Lucía Cañero Moslero

Octubre, 2026

UNIVERSIDAD
D
CÓRDOBA

Índice general

Índice de figuras **2**

1. Práctica 5 Arquitectura de Redes Neuronales **3**
1. Tarea 1 El problema de la no linealidad (Perceptrón Simple) 4
2. Tarea 2 Diseñando la capa oculta 5
3. Tarea 3 Funciones de activación: Sigmoide vs ReLU 8
4. Tarea 4 El reto de la caja negra (Dataset Dígitos) 10
5. Reto - Decisión técnica para producción 11
6. Conclusiones finales 12

Bibliografía **14**

Alicia Muriel Fernández
Lucía Cañero Moslero

Índice de figuras

1. Frontera de decisión del perceptrón simple. 4
2. Matriz de confusión del modelo lineal. 4
3. Frontera de decisión con una capa oculta de 2 neuronas. 6
4. Frontera de decisión con una capa oculta de 5 neuronas. 6
5. Frontera de decisión con una capa oculta de 20 neuronas. 7
6. Curva de pérdida (Loss) - Sigmoide. 8
7. Curva de pérdida (Loss) - ReLU. 8
8. Frontera de decisión - Sigmoide. 9
9. Frontera de decisión - ReLU. 9

Alicia Muriel Fernández
Lucía Cañero Moslero

Capítulo 1

Cuaderno de Laboratorio Práctica 5: Arquitectura de Redes Neuronales (Perceptrón Multicapa, MLP)

Objetivo

Comprender cómo la arquitectura de una red neuronal afecta a su capacidad de representación y generalización. En particular, estudiar por qué un modelo lineal falla en problemas no lineales, cómo influye el número de neuronas y capas ocultas, y qué papel juegan las funciones de activación (Sigmoide vs ReLU).

Material de partida

Se proporcionan dos conjuntos de datos y un script base (`practica5_mlp_alumnos.py`):

- Un dataset sintético no lineal (`make_moons`), en el que las clases no pueden separarse mediante una recta.
- Un dataset de clasificación de dígitos escritos a mano (basado en MNIST) para arquitecturas más profundas.

Nota: En este cuaderno se insertan las gráficas y tablas generadas automáticamente por el script en la carpeta `figures/` y `resultados/`.

Alicia Muriel Fernández
Lucía Cañero Moslero

1. Tarea 1 El problema de la no linealidad (Perceptrón Simple)

Configuración

Modelo usado: MLP sin capas ocultas (`hidden_layer_sizes=()`)
Métrica: Accuracy
Dataset: `make_moons` (ruido = 0.25)

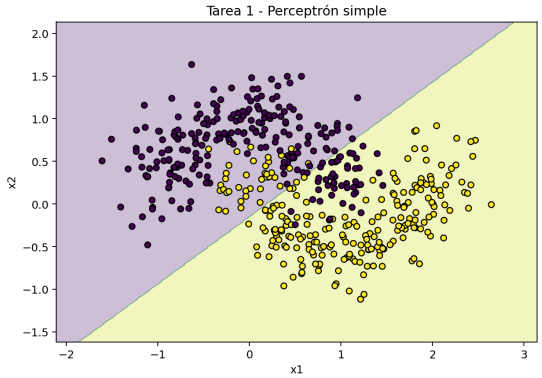


Figura 1: Frontera de decisión del perceptrón simple.

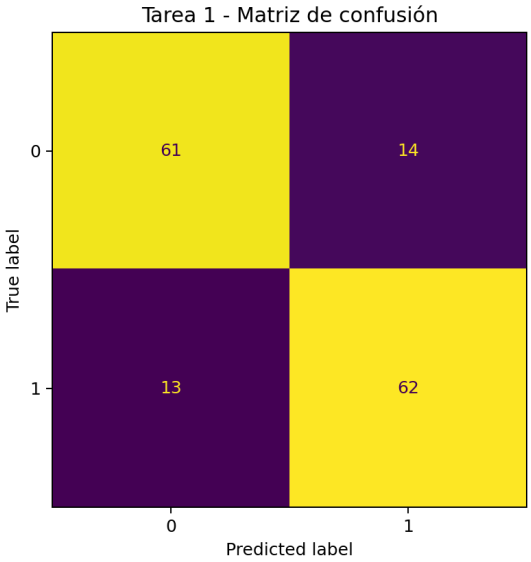


Figura 2: Matriz de confusión del modelo lineal.

Registro de resultados (Rellena con la terminal):

Modelo	Arquitectura	Activación	Accuracy Train	Accuracy Test
Perceptrón simple	()	relu	0.8286	0.8200

Análisis de los resultados:

- **¿Por qué la frontera aprendida es lineal y falla en este problema?** La frontera de decisión aprendida es lineal ya que el modelo empleado es un perceptrón sin capas ocultas. Al carecer de estas capas, la red neuronal no puede realizar transformaciones sucesivas, por lo que en la práctica actúa como un modelo lineal (equivalente a una regresión logística). Esto implica que

Alicia Muriel Fernández
Lucía Cañero Moslero

la predicción depende únicamente de una combinación lineal entre los pesos y las características de entrada. Por tanto, el modelo está limitado a separar el espacio mediante un único hiperplano, sin posibilidad de curvar la frontera para adaptarse a distribuciones más complejas.

- **Relación geométrica y capacidad expresiva:**
  El bajo rendimiento del modelo, aproximadamente el 82 %, no es un fallo de ejecución ni un problema de entrenamiento, sino una limitación derivada de su baja capacidad expresiva. La arquitectura seleccionada define el tipo de fronteras que el modelo es capaz de representar.
  Geometría de los datos: El conjunto de datos make_moons posee una topología no lineal, formada por dos formas de medialuna que se entrelazan.
  Limitaciones de la frontera: Una línea recta carece de la flexibilidad geométrica necesaria para separar estas dos estructuras. Cualquier recta que se dibuje sobre las medialunas atravesará inevitablemente zonas del espacio que pertenecen a la clase opuesta. Así, el modelo lineal falla porque su poder expresivo resulta insuficiente para capturar la verdadera distribución subyacente de los datos, evidenciando que cuando las clases presentan geometrías complejas, una línea recta no es suficiente.

2. Tarea 2 Diseñando la capa oculta

Configuración

Añadimos una única capa oculta variando el número de neuronas.
Configuraciones evaluadas: (2,), (5,) y (20,) neuronas
Registro de resultados:

Arquitectura	Accuracy Train	Accuracy Test	Iteraciones
(2,)	0.8429	0.8200	648
(5,)	0.8686	0.8400	726
(20,)	0.8600	0.8533	406

Alicia Muriel Fernández
Lucía Cañero Moslero

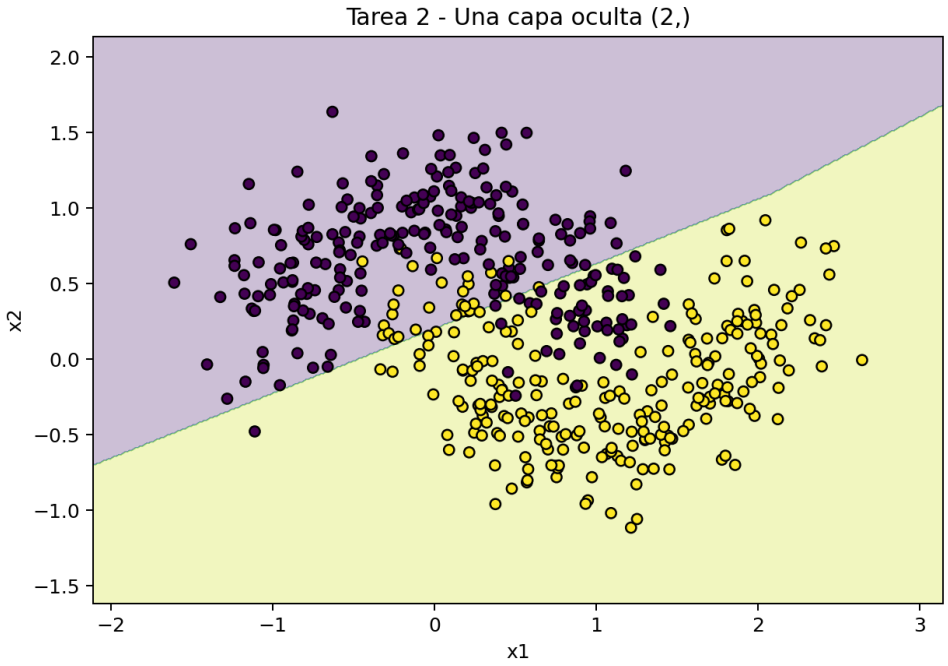


Figura 3: Frontera de decisión con una capa oculta de 2 neuronas.

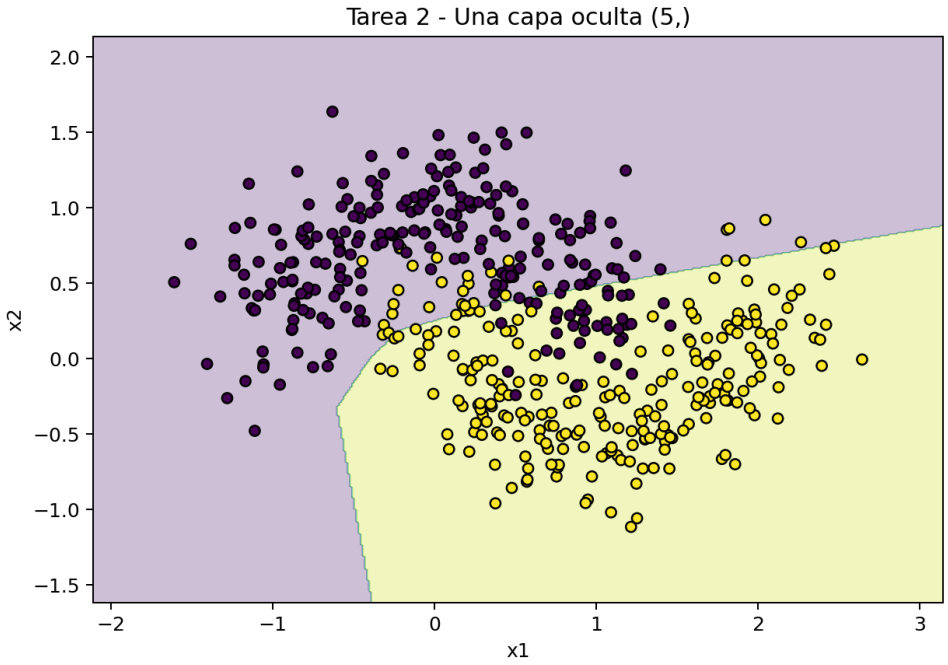


Figura 4: Frontera de decisión con una capa oculta de 5 neuronas.

Alicia Muriel Fernández
Lucía Cañero Moslero

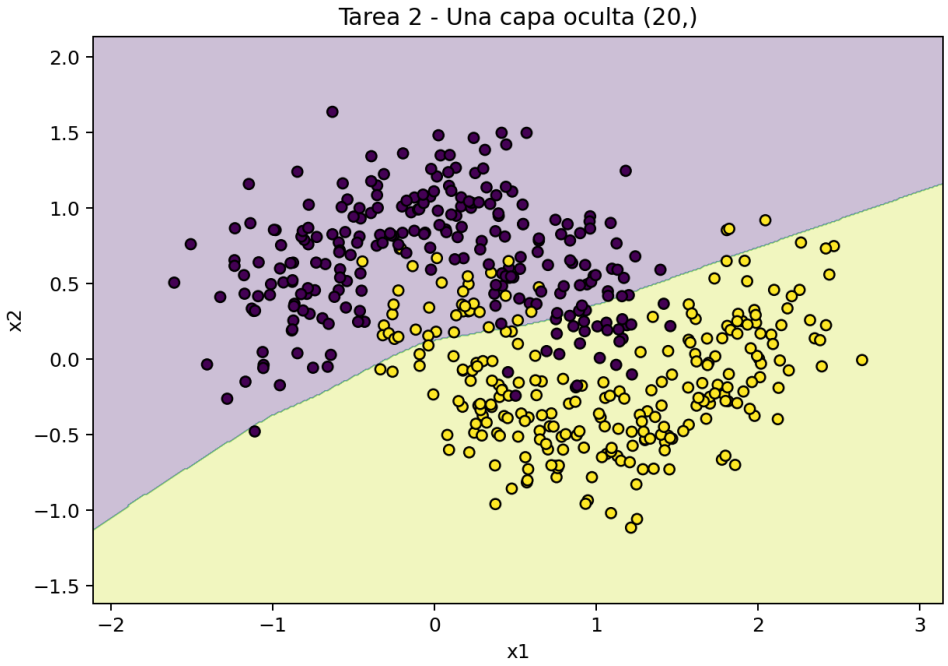


Figura 5: Frontera de decisión con una capa oculta de 20 neuronas.

Análisis comparativo:

- **Evolución visual de la frontera:**
  (Compara las tres figuras. Observa cómo con 2 neuronas la frontera está compuesta por pocas "rectas" unidas (todavía simple), y cómo con 20 neuronas envuelve las clases con mucha más precisión.)
  - *Arquitectura con 2 neuronas:* Los resultados muestran que, con pocas neuronas, la frontera de decisión sigue siendo deficiente. La línea de separación se compone de unos pocos segmentos rectos unidos de manera brusca, lo que evidencia que esta configuración resulta aún demasiado simple para ajustarse a la forma curva de las medialunas.
  - *Arquitectura con 5 neuronas:* Al incrementar la cantidad de neuronas en la capa oculta, el modelo adquiere mayor flexibilidad. La frontera se torna más suave y empieza a rodear las clases de forma más natural, indicando que la red logra captar mejor la estructura no lineal de los datos.
  - *Arquitectura con 20 neuronas:* Con este número de unidades, el modelo traza un límite mucho más detallado alrededor de los puntos. Sin embargo, puede observarse que, en ciertas zonas, la frontera se vuelve excesivamente irregular.

- **Underfitting vs. Overfitting:**
  (Identifica qué configuración parece sufrir *underfitting* (probablemente la de 2)

Alicia Muriel Fernández
Lucía Cañero Moslero

y si la de 20 muestra signos de *overfitting* (creando ”islas” o picos demasiado ajustados al ruido de entrenamiento).)
- *Subajuste (2 neuronas):* Esta configuración presenta subajuste porque el modelo es demasiado simple. Su limitada capacidad expresiva le impide aprender adecuadamente el patrón subyacente, lo que se traduce en un ajuste deficiente. El modelo se queda corto tanto en términos visuales como en métricas (84.29 % de precisión en entrenamiento).
- *Punto de equilibrio (5 neuronas):* Esta arquitectura parece lograr un buen balance. Capta la esencia geométrica del problema sin forzar formas extrañas, alcanzando un 86.86 % de rendimiento en entrenamiento.
- *Sobreajuste (20 neuronas):* Es importante recordar que un exceso de flexibilidad también puede resultar problemático. Aunque el modelo con 20 neuronas se ajusta estrechamente a los datos, el hecho de que empiece a generar ”islas” o picos para rodear puntos aislados es un indicio de que se adapta en exceso a los datos. Esta complejidad desmedida implica que el modelo comienza a memorizar el ruido presente en el conjunto de entrenamiento, lo que puede derivar en sobreajuste.

3. Tarea 3 Funciones de activación: Sigmoide vs ReLU

Configuración

Arquitectura fija: (20,) neuronas
Activaciones comparadas: `logistic` (Sigmoide) y `relu`

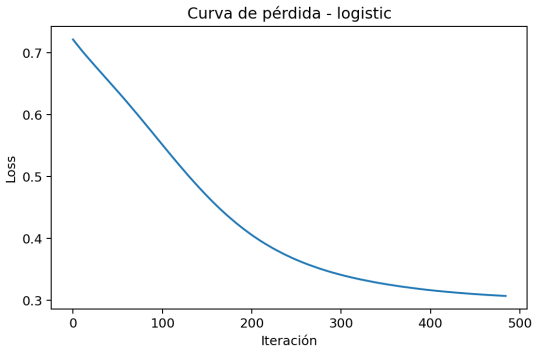


Figura 6: Curva de pérdida (Loss) - Sigmoide.

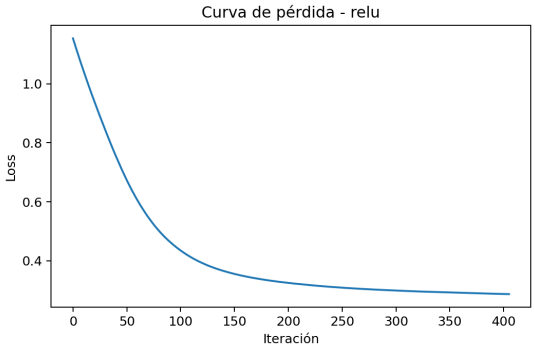


Figura 7: Curva de pérdida (Loss) - ReLU.

Alicia Muriel Fernández
Lucía Cañero Moslero

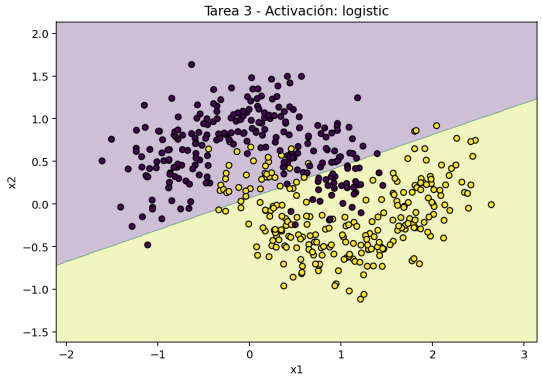


Figura 8: Frontera de decisión - Sigmoide.

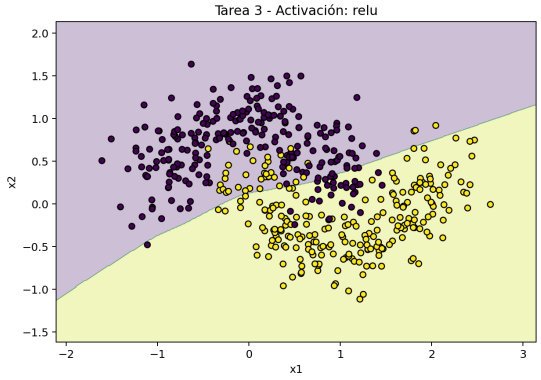


Figura 9: Frontera de decisión - ReLU.

Análisis de resultados:

- **¿Cuál de las dos funciones converge más rápido y presenta mayor estabilidad?**
  Al analizar los resultados experimentales y comparar las **curvas de pérdida (Loss) de las gráficas de arriba**, vemos que la red con **ReLU converge significativamente más rápido**. Esto se debe a que ReLU tiene 406 iteraciones, mientras que Sigmoide tiene 485. Además, ReLU alcanza un error final de pérdida menor, con un valor de 0.2870, en comparación con el 0.3071 de Sigmoide.

  Si observamos las gráficas, vemos que la curva de pérdida de ReLU baja de manera más pronunciada y estable hacia el mínimo. Esto ocurre porque la función ReLU tiene una derivada constante de 1 en su parte positiva, facilitando la actualización de los pesos de manera eficiente, sin que el gradiente desaparezca.

- **Aspecto de la frontera y saturación:**
  La función sigmoide reduce cualquier número real a un intervalo específico. Esto explica por qué **la gráfica de la frontera de decisión de la Sigmoide** muestra contornos muy suaves y redondeados. Sin embargo, esta compresión provoca el grave problema del **desvanecimiento del gradiente** (*vanishing gradient*): ante valores de entrada muy altos o muy bajos, las derivadas de la sigmoide se vuelven prácticamente planas (saturación), lo que detiene el aprendizaje de la red.

  Por el contrario, ReLU no se satura en los valores positivos, y simplemente envía los negativos a 0. **La gráfica de la frontera de decisión de ReLU**, que está formada por combinaciones de tramos lineales rectificados, genera

una división poligonal a trozos. Aunque puede parecer más abrupta, esta gráfica muestra que ReLU es capaz de capturar la estructura no lineal de los datos de manera más eficiente, sin sufrir problemas de saturación durante el entrenamiento.

4. Tarea 4 El reto de la caja negra (Dataset Dígitos)

Objetivo: Encontrar la mínima arquitectura razonable que alcance $\geq 95\,\%$ de *accuracy* en el dataset de dígitos.
Resumen de la exploración de arquitecturas:

Arquitectura	Acc. Train	Acc. Test	Iteraciones	$\geq 95\,\%$?
(20,)	1.0000	0.9644	276	Sí
(50,)	1.0000	0.9711	180	Sí
(100,)	1.0000	0.9800	147	Sí
(30, 15)	1.0000	0.9600	161	Sí
(50, 20)	1.0000	0.9756	131	Sí
(64, 32)	1.0000	0.9667	110	Sí

Análisis y decisión de diseño:

- **Justificación de la arquitectura elegida:**
  (Indica cuál has elegido como la "mínima razonable". Por ejemplo, si (50,) ya da 95 %, no hace falta usar (64, 32) que gasta más recursos computacionales.)

  Como vemos en la tabla de arriba, todas las arquitecturas evaluadas logran un ajuste perfecto en entrenamiento (*Acc. Train* de 1.0000) y superan el umbral del 95 % en el conjunto de prueba. Sin embargo, el diseño de una red neuronal consiste en buscar el equilibrio óptimo entre capacidad, generalización y simplicidad. Por eso, **la arquitectura seleccionada como mínima razonable es la `(20,)`**.

  Aunque la tabla muestra que el modelo `(100,)` obtiene el mejor resultado con un (98 % de *Acc. Test*), elegirlo supondría un gasto innecesario de recursos computacionales. Además, aumenta el riesgo de *overfitting* (sobreajuste), ya que las arquitecturas demasiado grandes tienden a memorizar los ejemplos de entrenamiento en lugar de generalizar los patrones reales. La configuración `(20,)` logra un 96.44 % de acierto en la prueba, lo que es suficiente para resolver el problema de manera sencilla.

Alicia Muriel Fernández
Lucía Cañero Moslero

- **Redes anchas frente a redes profundas:**
  Al observar la columna de *Iteraciones* en la tabla, vemos que las redes profundas (como `(50, 20)` o `(64, 32)`) necesitan menos ciclos para converger (131 y 110 iteraciones, respectivamente) que una red de una única capa como la `(20,)`, que necesita 276 iteraciones. Esto ocurre porque la profundidad permite a la red extraer características y combinaciones de forma jerárquica, acelerando la optimización geométrica.

  No obstante, la tabla también evidencia que, para este problema en concreto, añadir múltiples capas ocultas incrementa la complejidad del modelo sin mejorar significaticamente la precisión final (por ejemplo, la red profunda `(30, 15)` se queda en un 96 % de *Acc. Test*, rindiendo menos que la red simple de 20 neuronas). En este caso, una única capa oculta es suficiente para separar las clases con éxito, evitando modelos excesivamente complejos que puedan volverse inestables.

5. Reto - Decisión técnica para producción

Como parte de un equipo técnico, justifico las siguientes pautas de diseño:

1. **¿Cuándo tiene sentido usar una red sin capas ocultas?**
   Debemos usar un modelo sin capas ocultas únicamente cuando el problema sea linealmente separable. Como hemos indicado en la tarea 1, un MLP sin capas ocultas actúa como una regresión logística. Al carecer de la capacidad de realizar transformaciones sucesivas, solo podemos trazar un hiperplano. Por lo tanto, esta es la opción ideal si la relación entre características y clases es simple y lineal, ya que será computacionalmente más barato.

2. **¿Qué arquitectura recomendarías para un problema no lineal sencillo?**
   Para un problema no lineal sencillo sugerimos usar una arquitectura de una única capa oculta con pocas neuronas. Como comentamos en la tarea 2, una capa con 5 neuronas representan el punto de equilibrio. Con 2 neuronas el modelo sufre underfitting, mientras que con 5 consigue la flexibilidad geométrica justa para ajustarse a las clases de manera natural.

3. **Ventajas e inconvenientes de aumentar neuronas:**
   Ventajas: Aumenta la capacidad expresiva del modelo. La frontera de decisión se vuelve más flexible, suave y detallada. Esto le permite a la red captuarr estructuras no lineales más complejas y ajustarse mejor a los datos reales.
   Inconvenientes: Aumenta el riesgo de sobreajuste. Tal y como describimos

en la tarea 2 con la configuración de 20 neuronas, el modelo adquiere tanta capacidad de adaptación que comienza a memorizar el ruido presente en los datos de entrenamiento. Además, tiene un mayor coste computacional.

4. **Elección de función de activación general:**
   Como indicamos en la Tarea 3, la mejor elección es **ReLU** porque tiene **convergencia más rápida y estable**: alcanza un menor error final (0,287) en menos iteraciones (406 comparado con las 485 de sigmoide). Otra razón por la que ReLU es la mejor opción es porque **evita la saturación**: las derivadas de la función sigmoide tienden a valores cercanos a cero en los extremos, lo que paraliza el aprendizaje. Por el contrario, ReLU conserva un gradiente constante de 1 en su región positiva, esto facilita la actualización de los pesos de manera eficiente y permite trazar fronteras de decisión que representan bastante bien la no linealidad.

5. **£Cómo decidirías si una arquitectura es suficientemente buena?:**
   Una arquitectura se considera buena cuando encuentra el equilibrio óptimo entre capacidad, generalización y simplicidad. Como se justifica en la Tarea 4, no tratamos de buscar el 100 % de acierto a cualquier coste, sino que se considera "suficientemente buena" cuando supera la métrica objetivo en el conjunto de prueba empleando la configuración más simple posible.

6. **Criterio de parada para la complejidad:**
   Aplicamos el principio de parsimonia (Navaja de Ockham). Debemos detener la complejidad cuando:
   - Agregar más capas o neuronas apenas mejora los resultados en *test*.
   - El acierto en *train* llega hasta el 100 % pero en *test* se mantiene o incluso baja. Esto significa que el modelo ya no aprende patrones sino que memoriza datos.

6. Conclusiones finales

Tras el análisis de los resultados, se concluye que el perceptrón simple es insuficiente para problemas con geometrías complejas como make_moons, siendo necesarias las no linealidades que aportan las capas ocultas para transformar el espacio de características. Respecto al diseño de la capa oculta, el número de neuronas debe equilibrar expresividad y generalización: muy pocas provocan underfitting (por ejemplo, 2 neuronas) y muchas generan overfitting (20 neuronas), mientras que un valor intermedio (5 neuronas) consigue una frontera robusta.
En cuanto a las funciones de activación, ReLU supera claramente a la sigmoide, ya que acelera la convergencia, reduce el error final y evita el desvanecimiento del

gradiente.
Además, en la experimentación con el Dataset Digitos se confirma el principio de parsimonia: una arquitectura sencilla con una sola capa oculta es suficiente para superar la precisión deseada, mientras que modelos excesivamente profundos desperdician recursos y perjudican la generalización. En resumen, diseñar una red neuronal no es añadir capas sin criterio, sino buscar la arquitectura más simple capaz de resolver la verdadera complejidad del problema.

Bibliografía

[1] Sebastián Ventura and Christian Luna. Guión práctica 5 - arquitectura de redes neuronales (perceptrón multicapa, mlp), 2026. Asignatura: Introducción al Aprendizaje Automático. Accedido en abril 2026.

[2] Universidad de Córdoba. Práctica 5: Arquitectura de redes neuronales (perceptrón multicapa, mlp) (diapositivas), 2026. Asignatura: Introducción al Aprendizaje Automático (3ª Ingeniería Informática). Accedido en abril 2026.

[3] Sebastián Ventura Soto. Tema 6 - redes neuronales. (diapositivas de teoría), 2026. Asignatura: Introducción al Aprendizaje Automático. Accedido en abril 2026.

[4] Sebastián Ventura Soto. Tema 04 - ajuste fino y selección de modelos (diapositivas de teoría), 2026. Asignatura: Introducción al Aprendizaje Automático. Accedido en abril 2026.

[5] Sebastián Ventura Soto. Tema 07 - evaluación de clasificadores (diapositivas de teoría), 2026. Asignatura: Introducción al Aprendizaje Automático. Accedido en abril 2026.

[6] Sebastián Ventura Soto. Tema 05b - clasificación logística. (diapositivas de teoría), 2026. Asignatura: Introducción al Aprendizaje Automático. Accedido en abril 2026.